UNIVERSITE CHOUAIB DOUKKALI
ECOLE NATIONALE DES SCIENCES APPLIQUEES D'EL JADIDA

جامعة شعيب الدكالي
Université Chouaib Doukkali

Cybersécurité & Confiance Numérique (CCN)

Algorithmique et programmation avancée avec Python

Pr. Fatima zahra SALMAM
salmam.f@ucd.ac.ma

python™

1

Partie III :

- ✓Conteneurs standard: listes, dictionnaires, tuples et sets
- ✓ Les fichiers
- ✓ Les modules

32

Les modules: définition

- Les modules sont des programmes Python qui contiennent des fonctions que l'on est amené à réutiliser souvent (on les appelle aussi bibliothèques ou librairies).
- De nombreux modules qui effectuent une quantité phénoménale de tâches sont prédéfinis.
- Pour cette raison, prenez toujours le réflexe de vérifier si une partie du code que vous souhaitez écrire n'existe pas déjà sous forme de module.
- La plupart de ces modules sont déjà installés dans les versions standards de Python. Vous pouvez accéder à une documentation exhaustive sur le site de Python.

33

Les modules: Importation de modules

- Dans les parties précédentes de ce cours, nous avons rencontré la notion de module plusieurs fois. Par exemple, lorsque nous avons voulu tirer un nombre aléatoire :

```
>>> import random
>>> random . randint (0 , 10)
    4
```

- L'instruction import donne accès à toutes les fonctions du module **random**.
- La fonction **randint(0, 10)** du module **random** renvoie un nombre entier tiré aléatoirement entre 0 inclus et 10 inclus.

34

Les modules: Importation de modules

- L'utilisation de la syntaxe import module permet d'importer tout une série de fonctions organisées par «thèmes ».
- Par exemple, les fonctions gérant les nombres aléatoires avec **random** et les fonctions mathématiques avec **math**.
- Python possède de nombreux autres modules internes (c'est-à-dire présent de base lorsqu'on installe Python).
- Une méthode est une fonction un peu particulière :
  - Elle est liée à un objet par un point;
  - En général, elle agissait sur ou utilisait l'objet auquel elle était liée.

35

Les modules: Importation de modules

- Par exemple, la méthode **.format()** dans l'instruction **"{}".format(3.14)** utilise l'objet chaîne de caractères **"{}"** (auquel elle est liée) pour finalement renvoyer une autre chaîne de caractères **"3.14"**.
- Il existe un autre moyen d'importer une ou plusieurs fonctions d'un module :

```
>>> from random import randint
>>> randint (0 , 10)
    7
```

- À l'aide du mot-clé **from**, on peut importer une fonction spécifique d'un module donné. Remarquez bien qu'il est inutile de répéter le nom du module dans ce cas, seul le nom de la fonction en question est requis. On peut également importer toutes les fonctions d'un module :

```
>>> from random import *
```


36

Les modules: Importation de modules

- L'instruction from random import * importe toutes les fonctions du module **random**. On peut ainsi utiliser toutes ses fonctions directement, comme par exemple **shuffle()** qui permute une liste aléatoirement.
- Il est également possible de définir un alias (un nom plus court) pour un module :

```
>>> import random as rand
>>> rand.randint (0 , 10)
    6
```

- Dans cet exemple, les fonctions du module random sont accessibles via l'alias rand.

37

Les modules: Importation de modules

- Enfin, pour vider de la mémoire un module déjà chargé, on peut utiliser l'instruction del :

```
>>> import random
>>> random . randint (0 ,10)
      2
>>> del random
>>> random . randint (0 ,10)
      Traceback ( most recent call last ):
      File " < stdin >" , line 1 , in ?
      NameError : name 'random ' is not defined
```

- On constate alors qu'un rappel d'une fonction du module **random** après l'avoir vidé de la mémoire retourne un message d'erreur.

38

Les modules: Importation de modules

- Pour obtenir de l'aide sur un module, il suffit d'utiliser la commande help() :

```
>>> import random
>>> help(random)
```

- La commande help() est une commande plus générale permettant d'avoir de l'aide sur n'importe quel objet chargé en mémoire

```
>>> t = [1, 2, 3]
>>> help(t)
Help on list object:

class list(object)
 |  list() -> new list
 |  list(sequence) -> new list initialized from sequence's items
 |
 |  Methods defined here:
 |
 |  __add__(...)
 |      x.__add__(y) <==> x+y
 |
...
```


39

Les modules: Importation de modules

- Pour connaître d'un seul coup d'œil toutes les méthodes ou variables associées à un objet, utilisez la fonction **dir()** :

```
>>> import random
>>> dir(random)
['BPF', 'LOG4', 'NV_MAGICCONST', 'RECIP_BPF', 'Random', 'SG_MAGICCONST',
'SystemRandom', 'TWOPI', 'WichmannHill', '_BuiltinMethodType', '_MethodT
ype', '__all__', '__builtins__', '__doc__', '__file__', '__name__', '_ac
os', '_ceil', '_cos', '_e', '_exp', '_hexlify', '_inst', '_log', '_pi',
'_random', '_sin', '_sqrt', '_test', '_test_generator', '_urandom', '_wa
rn', 'betavariate', 'choice', 'expovariate', 'gammavariate', 'gauss', 'g
etrandbits', 'getstate', 'jumpahead', 'lognormvariate', 'normalvariate',
 'paretovariate', 'randint', 'random', 'randrange', 'sample', 'seed', 's
```


40

Les modules: Module sys (passage d'arguments)

- Le module **sys** contient des fonctions et des variables spécifiques à l'interpréteur Python lui-même. Ce module est particulièrement intéressant pour récupérer les arguments passés à un script Python lorsque celui-ci est appelé en ligne de commande.
- Par exemple, si nous créons un court script que l'on enregistrera sous le nom test.py :

```
1 import sys
2 print(sys.argv)
```

- Ensuite, dans un shell, nous exécutons le script test.py suivi de plusieurs arguments. Par exemple :

```
1 $ python test.py salut girafe 42
2 ['test.py', 'salut', 'girafe', '42']
```


41

Les modules: Module sys (passage d'arguments)

- Ensuite, dans un shell, nous exécutons le script test.py suivi de plusieurs arguments. Par exemple :

```
1 $ python test.py salut girafe 42
2 ['test.py', 'salut', 'girafe', '42']
```

- Le script affiche le contenu de la variable sys.argv. Cette variable est une liste qui contient tous les arguments de la ligne de commande, y compris le nom du script lui-même qu'on retrouve comme premier élément de cette liste dans sys.argv[0].
- On peut donc accéder à chacun des arguments du script avec sys.argv[1], sys.argv[2]. . .

42

Les modules: Module sys (passage d'arguments)

- La fonction **sys.exit()** est utile pour quitter un script Python. On peut donner un argument à cette fonction (en général une chaîne de caractères) qui sera renvoyé au moment où Python quittera le script.
- Par exemple, si vous attendez au moins un argument en ligne de commande, vous pouvez renvoyer un message pour indiquer à l'utilisateur ce que le script attend comme argument :

```
import sys

if len(sys.argv) != 2:
    sys.exit("ERREUR : il faut exactement un argument.")

print("Argument vaut : {}".format(sys.argv[1]))
```


43

Les modules: Module sys (passage d'arguments)

- Puis on l'exécute sans argument :

```
$ python test.py
ERREUR : il faut exactement un argument.
```

- Et avec argument:

```
$ python test.py 42
Argument vaut : 42
```

- Notez qu'ici on vérifie que le script possède deux arguments car le nom du script lui-même compte pour un argument (le tout premier).
- L'intérêt de récupérer des arguments passés dans la ligne de commande à l'appel du script est de pouvoir ensuite les utiliser dans le script.

44

Les modules: Module sys (passage d'arguments)

- Voici à titre d'exemple un script **compte_lignes.py** qui va prendre comme argument le nom d'un fichier puis afficher le nombre de lignes qu'il contient.

```
import sys

if len(sys.argv) != 2:
    sys.exit("ERREUR : il faut exactement un argument.")

nom_fichier = sys.argv[1]
taille = 0
with open(nom_fichier, "r") as f_in:
    taille = len(f_in.readlines())

print("{} contient {} lignes.".format(nom_fichier, taille))
```

- Supposons que dans le même répertoire, nous avons un fichier **zoo1.txt** dont voici le contenu :

```
girafe
tigre
singe
souris
```


45

Les modules: Module sys (passage d'arguments)

- et le fichier zoo2.txt qui contient :

```
poisson
abeille
chat
```

- Si on utilise maintenant le script **compte_lignes.py** :

```
$ python compte_lignes.py
ERREUR : il faut exactement un argument.
$ python compte_lignes.py zoo1.txt
zoo1.txt contient 4 lignes.
$ python compte_lignes.py zoo2.txt
zoo2.txt contient 3 lignes.
```

- Notre script est donc capable de :
  - Vérifier si un argument lui est donné et si ce n'est pas le cas d'afficher un message d'erreur.
  - D'ouvrir le fichier dont le nom est fourni en argument, de compter puis d'afficher le nombre de lignes.
- Par contre, le script ne vérifie pas si le fichier existe bien :

```
$ python compte_lignes.py zoo3.txt
Traceback (most recent call last):
  File "compte_lignes.py", line 8, in <module>
    with open(nom_fichier, "r") as f_in:
FileNotFoundError: [Errno 2] No such file or directory: 'zoo3.txt'
```


46

Les modules: Module os (interaction avec le SE)

- Le module **os** gère l'interface avec le système d'exploitation.
- La fonction **os.path.exists()** est une fonction pratique de ce module qui vérifie la présence d'un fichier sur le disque dur.

```
>>> import sys
>>> import os
>>> if os.path.exists("toto.pdb"):
...     print("le fichier est présent")
... else:
...     sys.exit("le fichier est absent")
...
le fichier est absent
```

- Dans cet exemple, si le fichier n'existe pas sur le disque, on quitte le programme avec la fonction **exit()** du module **sys**.

47

Les modules: Module os (interaction avec le SE)

- Améliorez le script **compte_lignes.py** dont le code a été donné précédemment de façon à ce qu'il renvoie un message d'erreur si le fichier n'existe pas. Par exemple, si les fichiers **zoo1.txt** et **zoo2.txt** sont bien dans le répertoire courant, mais pas **zoo3.txt**

```
import sys

if len(sys.argv) != 2:
    sys.exit("ERREUR : il faut exactement un argument.")

nom_fichier = sys.argv[1]
taille = 0
with open(nom_fichier, "r") as f_in:
    taille = len(f_in.readlines())

print("{} contient {} lignes.".format(nom_fichier, taille))
```


48

Les modules: Exercice

- Utilisez l'interpréteur Python pour affichez sur la même ligne les nombres de 10 à 20 (inclus) ainsi que leur racine carrée avec 3 décimales. Utilisez pour cela le module math avec la fonction sqrt(). Exemple :

```
10 3.162
11 3.317
12 3.464
13 3.606
[...]
```


49

Les modules: Exercice

- Affichez les nombres de 1 à 10 avec 1 seconde d'intervalle. Utilisez pour cela le module **time** et sa fonction **sleep().**
- Créez un script qui prend le nombre de seconde comme argument, puis exécutez-le dans un shell.

50

Les modules: Création de modules

- En Python, la création d'un module est très simple. Il suffit d'écrire un ensemble de fonctions (et/ou de constantes) dans un fichier, puis d'enregistrer ce dernier avec une extension **.py** (comme n'importe quel script Python).
- Par exemple, nous allons créer un module simple que nous enregistrerons sous le nom **message.py**

```
"""Module inutile qui affiche des messages :-)."""

DATE = 16092008


def bonjour(nom):
    """Dit Bonjour."""
    return "Bonjour " + nom


def ciao(nom):
    """Dit Ciao."""
    return "Ciao " + nom


def hello(nom):
    """Dit Hello."""
    return "Hello " + nom
```


Les modules: Utilisation de son propre module

- Pour appeler une fonction ou une variable de ce module, il faut que le fichier **message.py** soit dans le répertoire courant (dans lequel on travaille) ou bien dans un répertoire listé par la variable d'environnement PYTHONPATH de votre système d'exploitation.
- Ensuite, il suffit d'importer le module et toutes ses fonctions (et constantes) vous sont alors accessibles.
- Avec Windows, depuis un shell PowerShell, il faut taper la commande suivante :

  $env:PYTHONPATH += ";C:\chemin\vers\mon\super\module"

- Une fois cette manipulation effectuée, vous pouvez contrôler que le chemin vers le répertoire contenant vos modules a bien été ajouté à la variable d'environnement PYTHONPATH : : echo $env:PYTHONPATH

Les modules: Utilisation de son propre module

- Le chargement du module se fait avec la commande **import message**.
- Notez que le fichier est bien enregistré avec une extension .py et pourtant on ne la précise pas lorsqu'on importe le module.
- Ensuite, on peut utiliser les fonctions comme avec un module classique.

```
>>> import message
>>> message.hello("Joe")
'Hello Joe'
>>> message.ciao("Bill")
'Ciao Bill'
>>> message.bonjour("Monsieur")
'Bonjour Monsieur'
>>> message.DATE
16092008
```

Remarque:

- La première fois qu'un module est importé, Python crée un répertoire nommé _ _pycache_ _ contenant un fichier avec une extension .pyc qui contient le bytecode (le code précompilé du module).

Les modules: Utilisation de son propre module

Les docstrings:

- Lorsqu'on écrit un module, il est important de créer de la documentation pour expliquer ce que fait le module et comment utiliser chaque fonction.
- Les chaînes de caractères entre triple guillemets situées en début du module et de chaque fonction on les appelle docstrings (« chaînes de documentation » en français).
- Ces docstrings permettent notamment de fournir de l'aide lorsqu'on invoque la commande help() :

```
>>> help(message)

Help on module message:

NAME
    message - Module inutile qui affiche des messages :-).

FUNCTIONS
    bonjour(nom)
        Dit Bonjour.

    ciao(nom)
        Dit Ciao.

    hello(nom)
        Dit Hello.

DATA
    DATE = 16092008

FILE
    /home/pierre/message.py
```

Les modules: Utilisation de son propre module

Les docstrings:

- on peut aussi appeler l'aide pour une seule fonction :

```
>>> help(message.ciao)

Help on function ciao in module message:

ciao(nom)
    Dit Ciao.
```

- En résumé, les docstrings sont destinés aux utilisateurs du module. Leur but est différent des commentaires qui, eux, sont destinés à celui qui lit le code (pour en comprendre les subtilités).
- Une bonne docstring de fonction doit contenir tout ce dont un utilisateur a besoin pour utiliser cette fonction. Une liste minimale et non exhaustive serait :
  - ce que fait la fonction,
  - ce qu'elle prend en argument,
  - ce qu'elle renvoie.

55

Les modules: Utilisation de son propre module

Visibilité des fonctions dans un module :

- La visibilité des fonctions au sein des modules suit des règles simples :
  - Les fonctions dans un même module peuvent s'appeler les unes les autres.
  - Les fonctions dans un module peuvent appeler des fonctions situées dans un autre module s'il a été préalablement importé.
  - Par exemple, si la commande **import autremodule** est utilisée dans un module, il est possible d'appeler une fonction avec **autremodule.fonction().**

56

Les modules: Utilisation de son propre module

Module ou script ?

- Le module message ne contient que des fonctions et une constante. Si on l'exécutait comme un script classique, cela n'afficherait rien.

```
$ python message.py
$
```

- Cela s'explique par l'absence de programme principal, c'est-à-dire, de lignes de code que l'interpréteur exécute lorsqu'on lance le script.
- À l'inverse, que se passe-t-il alors si on importe un script en tant que module alors qu'il contient un programme principal avec des lignes de code ?
- Prenons par exemple le script message2.py suivant :

```
"""Script de test."""


def bonjour(nom):
    """Dit Bonjour."""
    return "Bonjour " + nom


# programme principal
print(bonjour("Joe"))
```

- Si on l'importe dans l'interpréteur, on obtient :

```
>>> import message2
Bonjour Joe
```


Les modules: Utilisation de son propre module

Module ou script ?

- Ceci n'est pas le comportement voulu pour un module car on n'attend pas d'affichage particulier (par exemple la commande import math n'affiche rien dans l'interpréteur).
- Afin de pouvoir utiliser un code Python en tant que module ou en tant que script, il faut utiliser la structure suivante :

```
"""Script de test."""


def bonjour(nom):
    """Dit Bonjour."""
    return "Bonjour " + nom


if __name__ == "__main__":
    print(bonjour("Joe"))
```

- À la ligne 9, l'instruction if _ _name_ _ == "_ _main_ _": indique à Python :
  - Si le programme message2.py est exécuté en tant que script dans un shell, le résultat du test if sera alors True et le bloc d'instructions correspondant (ligne 10) sera exécuté.

```
$ python message2.py
Bonjour Joe
```

  - Si le programme message2.py est importé en tant que module, le résultat du test if sera alors False (et le bloc d'instructions correspondant ne sera pas exécuté) :